Lead Intelligence Tool

Technical documentation · Assignment 2, Sales & Operations track · Abhijith K · October 2026

GitHub repository	github.com/kambhampatiabhijith-wq/lead-intelligence-tool
Replit project	replit.com/@kambhampatiabhi/lead-intelligence-tool
Stack	Python 3.13, Flask, SQLite, Groq API (openai/gpt-oss-120b), Chart.js

1. What the tool does

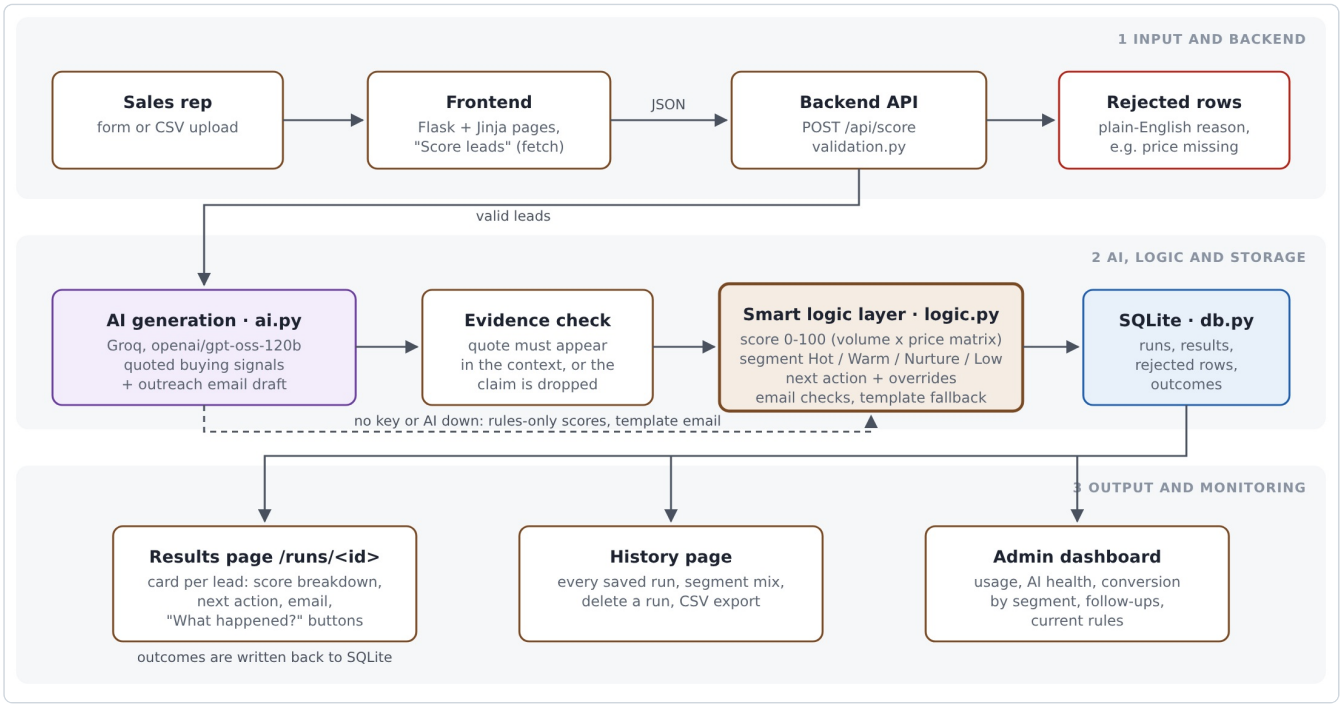
A B2B instant coffee manufacturer gets leads from enquiries, referrals, trade shows and cold outreach. A sales rep has to decide, for each one, how much it is worth, how fast to respond and what to say. Today that decision depends on the rep's memory and mood, and the reasons are rarely written down.

The Lead Intelligence Tool takes a lead (who the contact is, the company type, product, annual volume, indicative price, lead source and what the buyer said) and returns four things:

- a **score out of 100**, with every point explained
- a **segment**: Hot, Warm, Nurture or Low
- a **next action** for the rep
- a **personalised outreach email**

The AI reads the buyer's free text and drafts the email. It does not decide the score. Fixed rules in a separate logic layer turn its evidence into points, so the same lead always gets the same result, and a manager can see why.

2. System architecture



System architecture: input and backend, AI with logic and storage, output and monitoring

Each scoring run follows the flow the assignment asks for:

1. **User input.** The rep adds leads one at a time with a form, or uploads a CSV. Pending leads are stored in the database so they survive a restart.
2. **Frontend.** The Leads page sends the batch to the backend with `fetch()` as JSON. It never builds results itself.
3. **Backend API.** `POST /api/score` validates every row (`validation.py`). Bad rows are rejected with a plain-English reason, for example "Indicative price is missing; ask the buyer for a target price per kg."
4. **AI generation.** `ai.py` sends the valid leads to Groq in batches of three. For each lead the model returns buying

signals, each backed by a quote from the buyer's words, and an email draft.
5. **Smart logic layer.** `logic.py` checks the evidence, scores each lead, assigns the segment, picks the next action and checks the email. If the AI is unavailable, the rules still score every lead and a template email is used.
6. **Database.** `db.py` saves the run, every result, rejected rows and later outcomes in SQLite.
7. **Admin dashboard.** `/dashboard` reads the database to show usage, AI health and whether the segments predict real outcomes.
8. **Structured output.** Results appear as cards with a score table, badges and an email box. Raw JSON is never shown to the user.

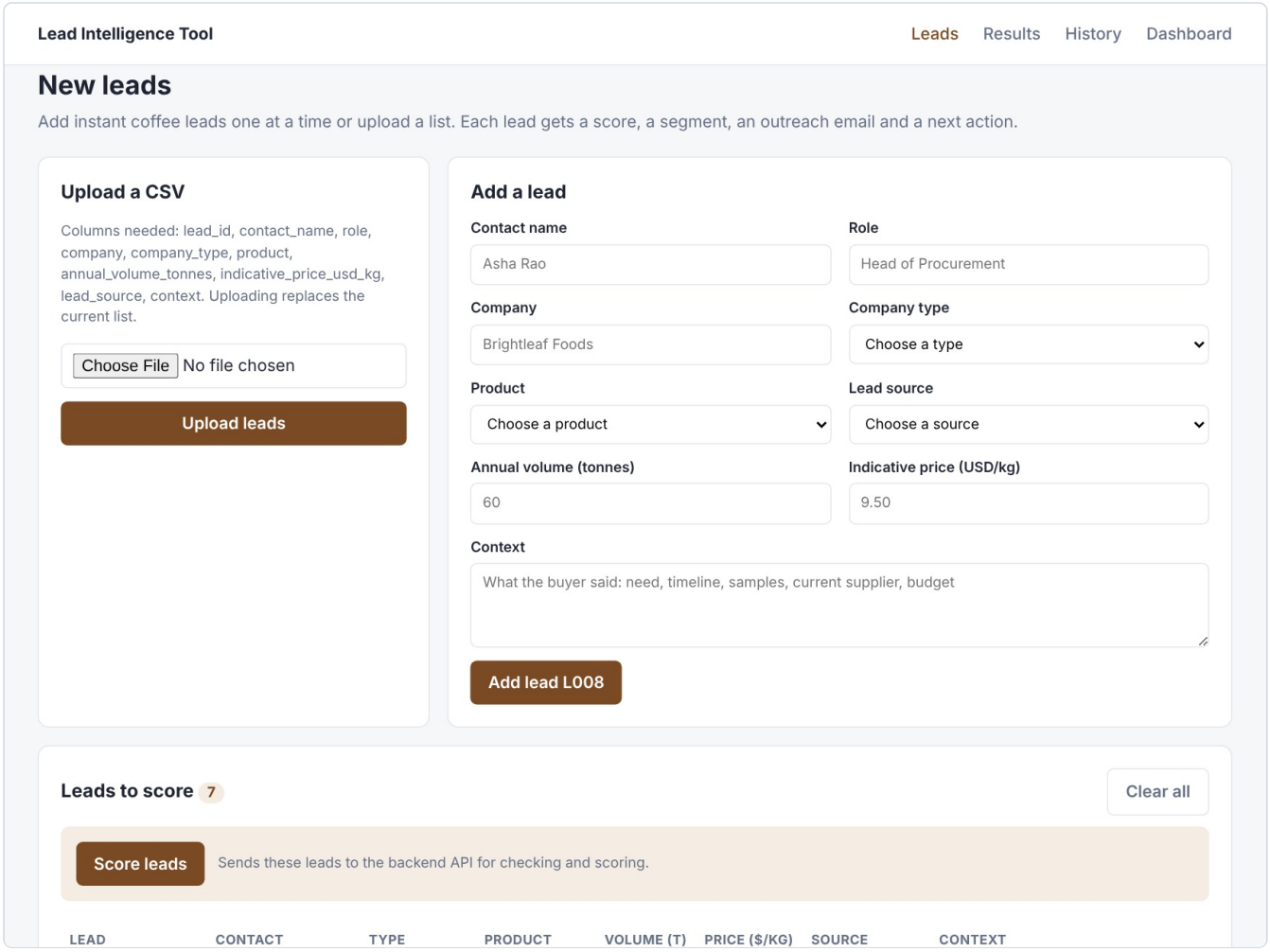
Project files

File	Role	Lines
`main.py`	Flask routes: pages, API endpoints, admin actions	212
`validation.py`	Input checks and clean-up	99
`ai.py`	Prompt, Groq call, retries, quote verification	239
`logic.py`	Scoring, segments, next actions, email checks	214
`db.py`	SQLite schema, saving and loading runs, dashboard numbers	363
`templates/`	Leads, Results, History and Dashboard pages	5 files
`static/style.css`	All styling	1 file

3. Frontend implementation

Screenshots in this document come from a local test run of the seven sample leads.

The frontend is four server-rendered pages built with Flask and Jinja templates, plus one small script.

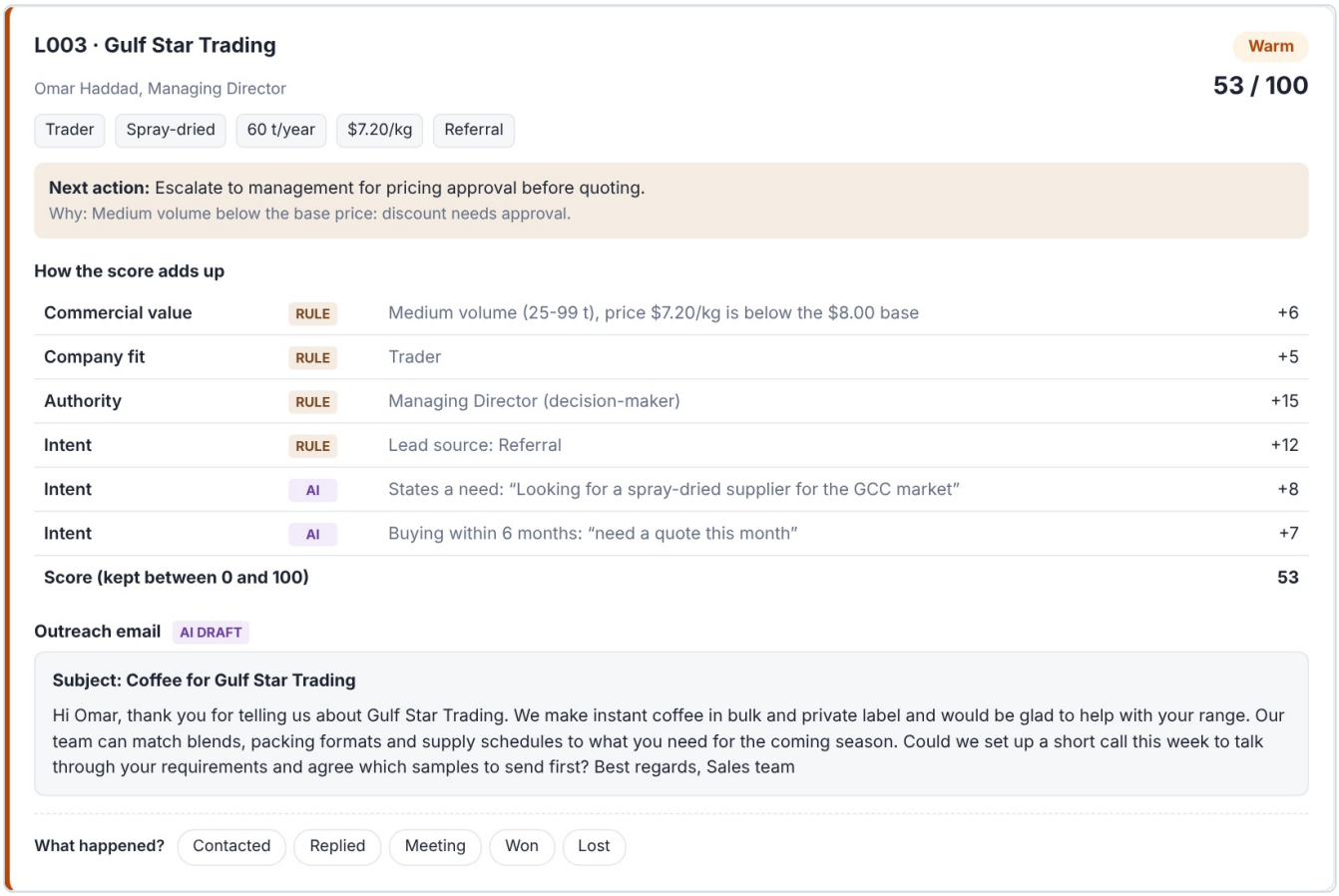


Leads page: CSV upload, the add-a-lead form and the list waiting to be scored

Leads page (`/`). Two ways in: a CSV upload that checks the file type, encoding and required columns, and a form with dropdowns for company type, product and lead source, so free-typed values can't break the scoring. Below them sits the list of leads waiting to be scored.

Score leads button. A short script posts the list to `/api/score` as JSON, shows "Scoring 7 leads…" while it waits, and on success opens the saved run. If the API returns an error, the message appears next to the button and the button is re-enabled.

Results page (`/runs/<id>`). One card per lead, ranked by segment and then score. Each card shows the badge and score, the facts, the next action and the reason for any override, a table showing how the score adds up (each row tagged RULE or AI), the outreach email (tagged AI draft or Template) and "What happened?" buttons for recording the outcome.



A result card. L003 is a Medium-volume lead below base price, so the next action is an escalation

History page (`/history`). Every saved run with its segment mix, average score and whether the AI ran, plus delete and CSV export.

Dashboard (`/dashboard`). Described in section 6.

The pages share one stylesheet with colour tokens. Segments keep the same colour everywhere (Hot red, Warm amber, Nurture blue, Low grey), and every colour comes with a text label, so nothing relies on colour alone. The layout was checked at phone width with no sideways scrolling.

4. Backend logic and the logic layer

API endpoints

Method and path	Purpose
`POST /api/score`	Validate, run AI, score, save. Returns JSON with run ID, results and rejected rows
`GET /api/runs/<id>`	Read a saved run back from the database as JSON
`POST /leads`, `POST /upload`, `POST /clear`	Manage the list of pending leads
`POST /results/<id>/outcome`	Record Contacted, Replied, Meeting, Won or Lost
`POST /runs/<id>/delete`, `GET /export.csv`	Admin: delete a run, export all scored leads

Validation

Required text fields must be present. Company type, product and lead source must come from the fixed lists. Volume must be a number above 0 and below 10,000 tonnes. Price is required and must fall between $2 and $60 per kg, since anything outside that range for instant coffee is almost certainly a typo. Context is capped at 2,000 characters, and duplicate lead IDs in one batch are rejected.

AI generation

The prompt asks for two things per lead. The first is **signals** from a closed list: states a need, buying within 6 months, asked for samples, tied to another supplier, no budget this year. Each needs a quote of 3 to 20 words copied from the context. The second is an **email** of 60 to 140 words that names the contact and company, refers to what they said, ends with one next step and mentions no prices.

Four safeguards sit around the call:

- **Quote check.** Every quote is matched against the original context (ignoring case, spacing and quote marks). A signal whose quote isn't found is dropped and listed on the card as "AI claim dropped".
- **Small batches.** Three leads per request. With all seven in one request, the reply sometimes got cut off and Groq rejected it as invalid JSON. Smaller batches fixed that and stay within free-tier token limits.
- **Retries.** Up to three attempts per batch. After a JSON error the next attempt drops strict JSON mode and parses the reply leniently. After a rate-limit error it waits before retrying.
- **Fallback.** If a batch still fails, only its leads skip the AI and the banner names them. With no API key, the whole run is scored by rules alone.

Scoring rules (out of 100)

Part	Max	How it is scored
Commercial value	40	Volume x price matrix (below)
Company fit	10	FMCG 10, Brand owner 9, Manufacturer 8, Retailer 6, Trader 5
Authority	15	Decision-maker (Founder, CEO, MD, Director, Head, VP) 15, influencer (Manager, Buyer, Procurement, Sourcing) 9, other 3
Intent	35	Lead source (Inbound 15, Referral 12, Trade show 8, Cold 3) plus AI signals (need +8, timeline +7, samples +5), capped at 35
Risk	-20	AI signals: tied to another supplier -10, no budget -10

Volume x price matrix. Volume only earns its full weight when the buyer's price is at or above our base price. A large buyer who wants a discount is worth less than the tonnage suggests.

Annual volume	At or above base price	Below base price
Large (100 t+)	40	10
Medium (25-99 t)	30	6
Small (5-24 t)	18	3
Micro (under 5 t)	8	0

Base prices are illustrative placeholders for this prototype: spray-dried $8.00, agglomerated $9.00, freeze-dried $14.00 per kg.

Segments and actions. Hot (70+): call within 24 hours and send samples with a spec sheet. Warm (45-69): send the email and book a discovery call this week. Nurture (25-44): follow up in 30 days. Low (under 25): no action this quarter.

Overrides, checked in this order:

1. If the lead is Hot or Warm, priced below base and Medium or Large volume: "Escalate to management for pricing approval before quoting."
2. If the lead is Hot or Warm and the contact isn't a decision-maker: "Ask for an introduction to the decision-maker before pitching."

Email checks. The AI draft is used only if it is 40 to 180 words, names the company and the contact, contains a clear next step (call, meeting or samples), mentions no price or discount, and has no placeholder in brackets. Otherwise a template written for the lead's segment is used, and the card says which check failed.

Test cases

The sample file has seven leads, each built to test one rule. With the AI finding the signals the context states:

Lead	Tests	Expected result
L001 Brightleaf Foods	Large volume above base, strong intent	Hot 100, call within 24 hours
L004 Coastline Beverages	Contact is a Purchase Manager	Warm 65, ask for an introduction
L003 Gulf Star Trading	Medium volume below base price	Warm 53, escalate for pricing approval
L002 Bean Street Coffee	Tied to another supplier until December	Warm 48
L006 Urban Bites Coffee	Large volume below base, no budget	Nurture 26, no escalation (not Hot or Warm)
L005 FreshMart Retail	Student enquiry, 3 tonnes	Low 20
L007 Nordic Brew Co	Price missing	Rejected by validation

Without the AI, the same file scores L001 80, L002 50, L004 50, L003 38, L006 36 and L005 20, because need and timeline only come from the context. In live runs the AI occasionally under-reports a signal (one run gave L001 92 because it listed the Q1 launch as a timeline but not as a need). The prompt was tightened to check every type, and the score table always shows exactly which signals counted.

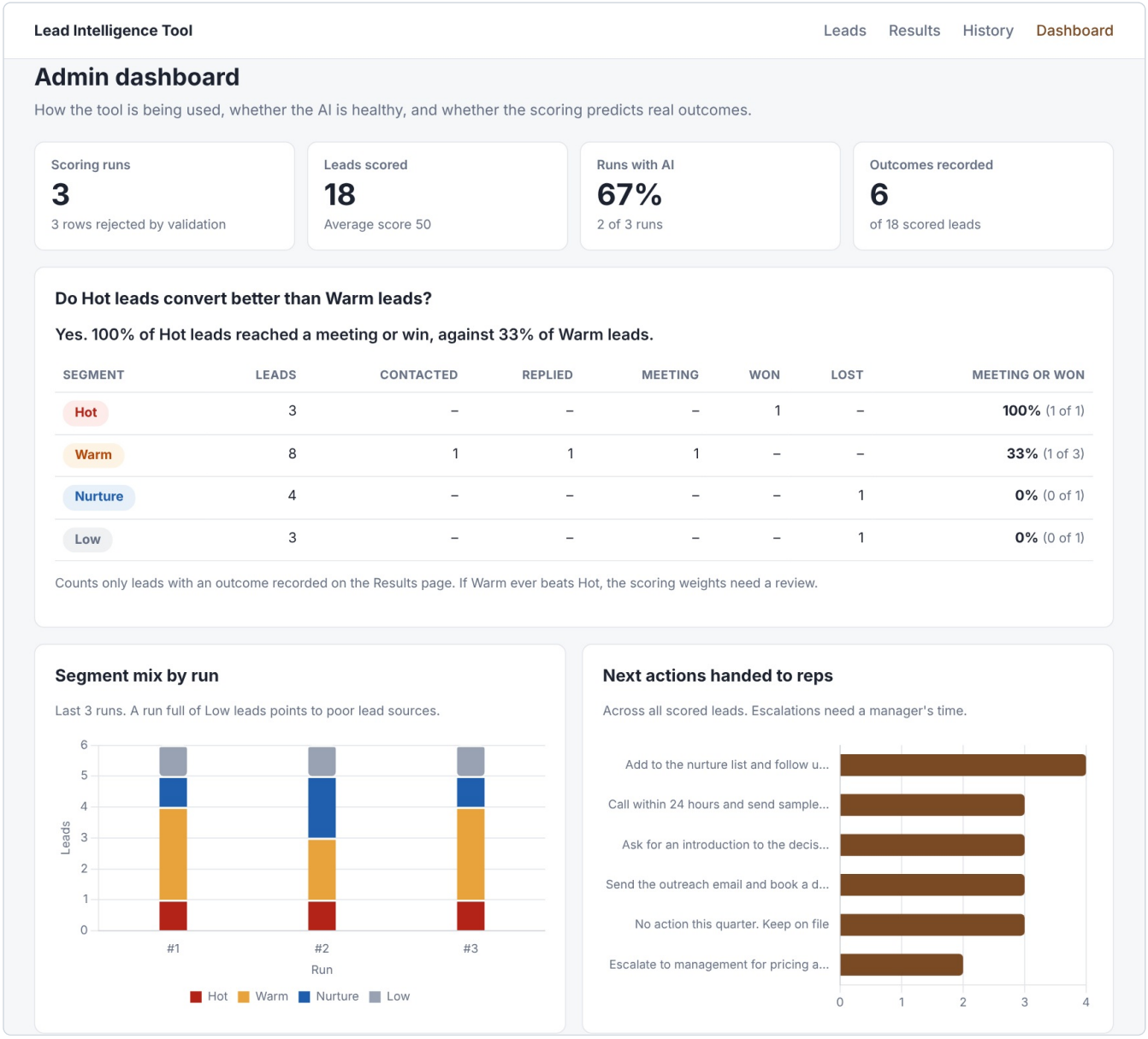
5. Database design

SQLite, in a single file (`leads.db`, excluded from Git). It needs no server, suits a single-team prototype and keeps the whole history queryable with plain SQL.

Table	Key columns	Purpose
`pending_leads`	position, lead fields	Leads waiting on the Leads page
`runs`	id, created_at, received, valid_count, rejected_count, ai_used, ai_provider, ai_model, ai_message	One row per scoring run
`results`	id, run_id → runs, lead fields, score, segment, next_action, action_reasons, parts, ai_dropped, email_subject, email_body, email_source, email_problems	One row per scored lead
`rejected`	id, run_id → runs, lead_id, company, errors	Rows that failed validation
`outcomes`	result_id → results (primary key), outcome, updated_at	What happened after outreach

A run has many results and many rejected rows. A result has at most one outcome, and recording a new one replaces the old one. Lists such as the score breakdown are stored as JSON text, so a saved run can be rebuilt exactly as it first appeared, even if the scoring rules change later. Times are stored in UTC and shown in India time.

6. Admin dashboard



Admin dashboard: KPIs, conversion by segment and the two charts. Below these sit follow-ups, AI health and the current rules

The dashboard answers the questions a sales manager would ask:

- **Is the tool being used?** Scoring runs, leads scored, rows rejected and outcomes recorded.
- **Does the scoring work?** A table of outcomes by segment, with the share of leads that reached a meeting or a win. Above it, a one-line verdict: “Yes. 100% of Hot leads reached a meeting or win, against 33% of Warm leads.” If Warm ever beats Hot, it says the weights need a review.
- **Where are leads coming from?** A stacked bar chart of the segment mix for the last 12 runs.
- **What are reps being told to do?** A bar chart of next actions, so a manager can see how many escalations are waiting for them.
- **Who needs chasing?** Hot and Warm leads from the latest run with no outcome yet, each linked to its card.
- **Is the AI healthy?** The share of runs where the AI ran, claims dropped for missing quotes, emails that fell back to the template, and a list of runs with AI problems and the error message.
- **What are the rules?** The current matrix, base prices and point values, read from `logic.py`, so the dashboard can never disagree with the code.

The History page provides the management actions: deleting a run and exporting every scored lead as CSV.

7. Git commit progression

The repository has 17 commits. Each build stage was tested before it was committed.

Stage	Commit	What it added
Setup	Initial commit; Add files via upload; Rename README_3.md to README.md	Repo with Python .gitignore, README with the scoring design, sample_leads.csv
Setup	Set up Replit config for a Python Flask app	`.replit`, requirements.txt, ignored folders
2 Frontend	Add lead input form and CSV upload	Form, CSV upload, leads table, base layout
3 Backend	Add backend scoring API with input validation	`POST /api/score`, validation.py, results page
4 AI	Add AI signal extraction and outreach email drafts	ai.py with Groq, quote checks, email drafts
5 Logic	Add files via upload; Add files via upload; Yes	logic.py, the new results page and the API wiring, uploaded through GitHub after long Shell pastes kept breaking
5 Logic	Show scores, segments and next actions on the results page; Add styles for score cards and segment badges	Results page in place, segment styles
5 Fixes	Make AI calls more reliable…; Ask the AI to check every signal type…	Batches of three, retries, partial fallback, prompt fix
6 Database	Save runs, results and outcomes in SQLite and add a History page	db.py, saved runs, outcomes, History, GET API
7 Dashboard	Add admin dashboard, run deletion and CSV export	/dashboard, delete, export, mobile fixes
8 Docs	Add documentation and update README	This document

Four commits have unclear names (“Add files via upload” three times and “Yes”). They came from uploading files through GitHub’s web page when pasting long code into the Replit Shell kept cutting it off. The table above records what each one contains.

8. Testing and limitations

How it was tested. Each stage was run against the seven sample leads before committing, using Flask’s test client with simulated AI replies. These included an invented quote (which was dropped), a vague email that quoted a price (which was replaced by the template), a JSON error and a rate-limit error (which were retried), and a missing API key (rules only). Live runs on Replit used the Groq free tier. Pages were checked in a browser at desktop and phone widths.

Limitations.

- Base prices and point values are illustrative. A real deployment would set them from current price lists and tune the weights against recorded outcomes, which is what the dashboard’s conversion table is for.
- The AI does not report signals the same way every time. Rules and quote checks limit the damage, and the score table makes any gap visible, but the same lead can score a few points differently between runs.
- The free Groq tier limits tokens per minute, so a very large CSV can hit rate limits. The app retries and then falls back for the affected leads only.
- On Replit’s published (autoscale) deployment the SQLite file is not permanent and can reset when the app restarts. For production, the same tables would move to a hosted database such as Replit’s PostgreSQL.
- There is no login. Anyone with the link can see the dashboard. For real use, the admin pages would sit behind sign-in.